MLA, MHA, Linear Attention, and Hybrid Models

Deep Dive into SGLang — Chapter 11

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that makes the cache plural.

- Until now the KV cache was one object: per-position key/value rows charged to a request.
- One request can now reach layers that store full KV rows, latent rows, sliding-window state, or recurrent state.
- SGLang must keep scheduling, memory accounting, and layer semantics aligned while the meaning of "attention state" changes under one request row.

Goal: by the end you can look at any layer and say which state object is stored, which positions are visible, and which metadata keeps the output attached to the scheduler row.

Roadmap

The attention-state problem

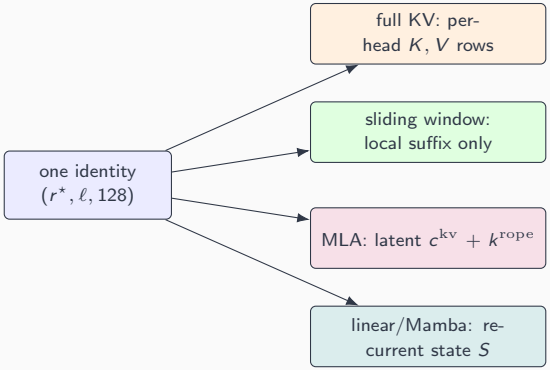
The families and the plan

The cross-cutting contract

The attention-state problem

One request, four kinds of memory

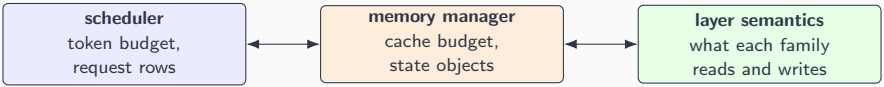
The running request is one admitted `google/gemma-4-E4B-it` chat request, written $r^\star$, held in decode at logical position 128.



four payloads, one owner

The request/layer/position identity is invariant; only the stored payload and its metadata change.

The contract: three accounts stay in agreement



- A family may change full KV rows, latent rows, sparse indices, windows, or recurrent state.
- The scheduler still charges that state to *one* request row; positions keep their request-local meaning.

Plurality of state, singularity of ownership — that pairing is the spine of the chapter.

The families and the plan

Four tricks, read as state consequences

Each technique just stores *less* while reading the same logical history.

MLA absorption move the key expansion to the query side, compare against a latent row, expand the value *after* the weighted sum.

NSA a bounded sparse top-k view over the same resident causal history.

Linear attention a recurrence: accumulated numerator/normalizer state (H, z) for a feature map ϕ.

Hybrid plans one checkpoint composes several families, layer by layer.

Ask of each: which state object is stored, and does it still read the scheduled tokens?

The hybrid layer plan: the central organizing object

Hybrid layer plan $\pi(\ell) = (f_\ell, p_\ell)$
A model-level map from layer id to *state family* f_ℓ and *backend path* p_ℓ.

Demonstration plan π_{demo} (a teaching object, not Gemma's real tower):

Layer	Family	Persistent state object
0	GQA	ordinary KV rows, group map g
1	sliding window	visible local-window KV state
2	MLA	latent row + positional key
3	linear	recurrent S^{lin} snapshot

Treating every layer as full KV overcounts some and passes the wrong metadata to others.

Why the chapter exists: capacity diverges by family

Same 2048 retained positions, 32 layers, 8 KV heads, FP16:

$$\underbrace{2048 \cdot 32 \cdot 8 \cdot (128{+}128) \cdot 2}_{\text{full KV}} = 268{,}435{,}456 \text{ B} \approx 256 \text{ MiB},$$

$$\underbrace{2048 \cdot 32 \cdot (512{+}64) \cdot 2}_{\text{MLA: } r_{\mathrm{kv}}=512,\ d_{\mathrm{rope}}=64} = 75{,}497{,}472 \text{ B} \approx 72 \text{ MiB}.$$

The scheduler cannot treat all attention families as one KV-byte formula. A sliding-window layer would replace 2048 by its window; a linear layer replaces the table with registered recurrence state.

From concept to code: a hybrid plan is real code

Kimi Linear chooses the layer object *per layer id*.

from `python/sglang/srt/models/kimi_linear.py`

```
if config.is_kda_layer(layer_idx):
    self.self_attn = KimiDeltaAttention(
        layer_idx=layer_idx,
        hidden_size=config.hidden_size,
        config=config, ...)
else:
    self.self_attn = KimiMLAAttention(   # = DeepseekV2AttentionMLA
        layer_id=layer_idx, ...)
```

The plan is not a label: `is_kda_layer` decides which layer object exists, which weights load, and which state family the request carries at that layer.

The cross-cutting contract

The invariant the whole chapter defends

Attention-family state alignment
A hybrid step is valid only when request identity, layer family f_ℓ, forward mode, capacity accounting, state lifetime, and batch transforms stay aligned for every state object $S_{r,\ell}$ it reads or writes.

- The dangerous bug keeps tensor **shapes plausible** while changing the attention meaning.
- An MLA latent row read as expanded K/V; recurrence read as a KV span; a padded linear call writing state for dummy tokens.

Shape compatibility is weaker than semantic compatibility.

What to carry forward

1. Attention-family support is a **state-management** problem before it is a kernel-selection problem.
2. One identity, **four payloads**: full KV, latent (MLA/NSA), recurrent (linear/Mamba), windowed.
3. The **hybrid layer plan** $\pi(\ell)$ is the central object; $S_{r,\ell}$ is what each request carries per layer.
4. Capacity follows the **family**: positions$\times$KV-heads, or latent rank, or window, or registered recurrence shape.
5. Every family keeps **row ownership** as one contract.

Chapter 10 chooses the backend; this chapter records what the state means before a backend sees it. Next: Chapter 12 turns these state families into a constructed `ForwardBatch`.